UNIDAD DIDÁCTICA · CLASE GUIADA

Métodos en Java

Aprende a organizar, reutilizar y entender tu código

Métodos void | Parámetros | return | Retos prácticos

Con ejemplos resueltos, explicados paso a paso, y su salida por consola

¿Qué aprenderemos hoy?

1 Qué es un método

Y por qué hace que nuestros programas sean más cortos, ordenados y fáciles de cambiar.

2 Declarar y llamar

Cómo se escribe un método (declaración) y cómo se pone en marcha (llamada).

3 Parámetros

Cómo enviar datos a un método para que trabaje con valores distintos cada vez.

4 void frente a return

Métodos que solo hacen una tarea y métodos que, además, devuelven un resultado.

5 Combinar lo aprendido

Métodos junto con if / else, bucles for y lectura de datos con Scanner.

6 Errores y práctica

Los fallos más habituales, cómo detectarlos y tres prácticas guiadas.

Antes de empezar: lo que ya sabemos

Variables

Guardan datos para usarlos después.

```
int edad = 20;
```

Scanner

Lee lo que escribe el usuario.

```
teclado.nextLine();
```

if / else

Elige qué hacer según una condición.

```
if (edad >= 18) {...}
```

for / while

Repite instrucciones varias veces.

```
for (int i=1; i<=5; i++)
```

Hasta ahora escribíamos todo dentro de main. **Cuando el programa crece, main se convierte en una lista enorme y difícil de leer. Hoy aprenderemos a ORGANIZAR ese código en piezas con nombre: los métodos.**

01

¿Qué es un método?

Qué es, para qué sirve y cómo se ejecuta paso a paso.

¿Qué es un método?

1. **Es un bloque de código con nombre.** Agrupa varias instrucciones bajo un nombre que describe lo que hacen, por ejemplo calcularMedia o mostrarMenu.

2. **Realiza una tarea concreta.** Cada método debería encargarse de UNA sola cosa. Si hace demasiadas, conviene dividirlo en varios.

3. **Se escribe una vez y se usa muchas.** Cuando necesitamos esa tarea, solo tenemos que «llamarlo» por su nombre, sin volver a escribir sus instrucciones.

4. **¡Ya los conoces!** main es un método. println() y nextLine() también son métodos, aunque los escribió otra persona.



Piensa en una receta de cocina

Tiene un **nombre**: «Tortilla de patatas».

Tiene unos **pasos**: pelar, freír, batir…

La escribes **una vez** y la cocinas **cuantas veces quieras**.

Un método es la receta; llamarlo es ponerse a cocinar.

¿Por qué usar métodos? Compara

✗ SIN MÉTODOS: CÓDIGO REPETIDO

```
System.out.println("==========");
System.out.println("   MENÚ");
System.out.println("==========");
// ... más código ...
System.out.println("==========");
System.out.println("   MENÚ");
System.out.println("==========");
```

✓ CON MÉTODOS: SE ESCRIBE UNA VEZ

```
mostrarCabecera();
// ... más código ...
mostrarCabecera();

public static void mostrarCabecera() {
    System.out.println("==========");
    System.out.println("   MENÚ");
    System.out.println("==========");
}
```



¿Y si mañana queremos cambiar «MENÚ» por «OPCIONES»? Sin métodos hay que buscar y corregir cada copia (y es fácil olvidar alguna). Con métodos, se cambia en un único lugar y todas las llamadas se actualizan solas.

Ventajas de trabajar con métodos

Evitan repetir código

Escribimos una tarea una sola vez y la reutilizamos tantas veces como haga falta.

Facilitan los cambios

Si hay que corregir o mejorar algo, se hace en un solo sitio. Menos trabajo y menos errores.

Hacen el código legible

Un nombre como calcularMedia() explica qué ocurre sin necesidad de leer cada línea.

Divide y vencerás

Un problema grande se divide en tareas pequeñas, cada una en su método, más fáciles de programar y de probar.

Primer método: saludar

Saludo.java

```java
public class Saludo {

    public static void main(String[] args) {
        saludar();     // LLAMADA
    }

    public static void saludar() {   // DECLARACIÓN
        System.out.println("¡Hola!");
    }
}
```

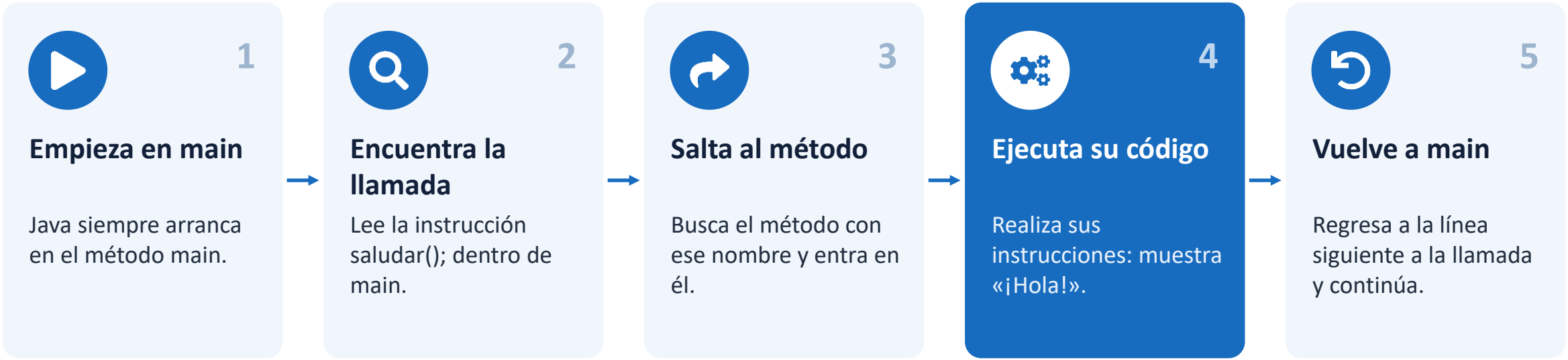
1. **La clase** Saludo contiene todo el programa: main y nuestro método.
2. **main** es el punto de partida. Java siempre empieza a ejecutar por aquí.
3. **saludar();** es la LLAMADA: le dice a Java «ejecuta ahora el método saludar».
4. **La declaración** public static void saludar() { … } describe QUÉ hace el método.

▶ SALIDA POR CONSOLA

```
¡Hola!
```

saludar() está fuera de main, pero dentro de la clase.

¿Qué ocurre al ejecutar? Paso a paso



Definir un método NO significa ejecutarlo. Escribir el método es como guardar una receta en el libro: hasta que nadie decide cocinarla, no pasa nada. Si nunca llamamos a saludar(), nunca veremos «¡Hola!» en pantalla.

Llamar varias veces al mismo método

```java
public static void main(String[] args) {
    saludar();
    saludar();
    saludar();
}

public static void saludar() {
    System.out.println("Bienvenido a Java");
}
```

▶ SALIDA POR CONSOLA

```
Bienvenido a Java
Bienvenido a Java
Bienvenido a Java
```

1. **Cada llamada ejecuta el método completo,** de la primera a la última instrucción. Tres llamadas = tres mensajes.
2. **Las llamadas se ejecutan en orden,** de arriba abajo, igual que cualquier otra instrucción de main.
3. **Lo que importa es el orden de las llamadas,** no el orden en que están escritos los métodos dentro de la clase.
4. **Un solo cambio basta:** si modificamos el texto dentro de saludar(), cambian los tres mensajes.

Anatomía de un método

```
public static void saludar() { … }
```

`public`

Acceso público: el método puede usarse también desde otras clases. De momento, lo escribiremos siempre.

`static`

Permite llamarlo directamente desde main sin crear objetos. Lo entenderás al estudiar clases; por ahora, siempre static.

`void`

Tipo de retorno. void significa «vacío»: el método no devuelve ningún resultado.

`saludar`

El nombre. Empieza en minúscula y suele ser un verbo: mostrarMenu, calcularTotal.

`( )`

Lista de parámetros: los datos que el método necesita recibir. Vacía = no necesita datos.

`{ }`

El cuerpo: las instrucciones que se ejecutan cada vez que se llama al método.

Cómo dar nombre a un método

1. **Empieza por un verbo:** el método HACE algo. mostrar, calcular, pedir, comprobar, convertir…

2. **Usa camelCase:** primera palabra en minúscula y cada palabra siguiente con mayúscula inicial: calcularPrecioFinal.

3. **Sé descriptivo:** quien lea el nombre debe adivinar qué hace sin mirar su código.

4. **Sin espacios, tildes ni símbolos raros:** ni empezar por número. Evita la ñ y las tildes para no tener problemas.

✗ Evita	✓ Mejor así
`m1()`	`calcularMedia()`
`hacerCosas()`	`mostrarMenu()`
`Sumar()`	`sumar()`
`calcular_total()`	`calcularTotal()`
`2numeros()`	`sumarDosNumeros()`

02

Parámetros

Cómo enviar datos a un método para que sea flexible y reutilizable.

Un método con una operación

```java
public static void mostrarSuma() {
    int numero1 = 8;
    int numero2 = 5;

    int resultado = numero1 + numero2;
    System.out.println(resultado);
}

// Desde main:
mostrarSuma();
```

1. **Crea dos variables** numero1 y numero2 con los valores 8 y 5.
2. **Calcula la suma** y la guarda en la variable resultado.
3. **Muestra el resultado** por pantalla con println: 13.
4. **Variables locales:** numero1, numero2 y resultado solo existen DENTRO del método. Desde main no se pueden usar.

▶ SALIDA POR CONSOLA

```
13
```



Funciona… pero siempre suma 8 + 5.

El problema de los valores fijos

1. **Si los valores están dentro del método,** siempre trabaja con los mismos datos y siempre da el mismo resultado.

2. **¿Y si queremos sumar 4 + 6, o 100 + 250?** Tendríamos que escribir un método nuevo para cada pareja de números. ¡Imposible!

3. **La solución:** dejar «huecos» en el método para que quien lo llame decida qué datos enviar.

4. **Esos huecos se llaman PARÁMETROS.** Gracias a ellos, un mismo método sirve para muchos datos diferentes.



Imagina una calculadora…

…que solo sabe sumar 8 + 5. No serviría para nada. Una calculadora útil tiene huecos donde TÚ escribes los números:

```
sumar( __ , __ )
```

Un método con parámetros es más flexible y reutilizable.

Métodos con parámetros

```java
public static void saludarPersona(String nombre) {
    System.out.println("Hola, " + nombre);
}

public static void main(String[] args) {
    saludarPersona("Ana");
    saludarPersona("Luis");
}
```

▶ SALIDA POR CONSOLA

```
Hola, Ana
Hola, Luis
```

`String nombre` es el parámetro: una variable que se rellena cada vez que se llama al método. **`"Ana"`** y **`"Luis"`** son los argumentos.

¿QUÉ HACE JAVA EN CADA LLAMADA?



Parámetros y argumentos: no son lo mismo

PARÁMETRO

Aparece en la DEFINICIÓN del método.
Es una variable con tipo y nombre: el «hueco» que espera un dato.

```
void saludarPersona(String nombre)
```

Como en un formulario: El campo vacío del formulario: «Nombre: _____»

ARGUMENTO

Aparece en la LLAMADA al método.
Es el valor concreto que enviamos para rellenar el hueco.

```
saludarPersona("Ana");
```

Como en un formulario: Lo que escribes en ese campo: «Nombre: Ana»

Regla de oro: los argumentos deben coincidir con los parámetros en número, orden y tipo.

Un método con dos parámetros

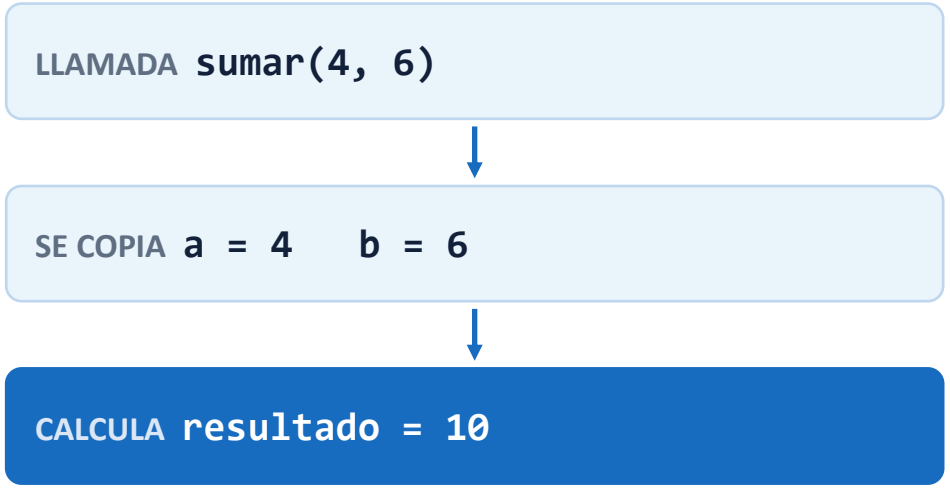
```java
public static void sumar(int a, int b) {
    int resultado = a + b;
    System.out.println("Suma: " + resultado);
}

public static void main(String[] args) {
    sumar(4, 6);
    sumar(10, 20);
}
```

▶ SALIDA POR CONSOLA

```
Suma: 10
Suma: 30
```

EL PRIMER ARGUMENTO VA AL PRIMER PARÁMETRO



1. **Separados por comas,** tanto en la definición como en la llamada.
2. **Cada parámetro lleva su tipo:** (int a, int b) ✓ (int a, b) ✗

Tres reglas al pasar argumentos

```
public static void mostrarDatos(String nombre, int edad)
```

1 · Mismo número

Si el método pide 2 datos, hay que enviar exactamente 2.

✓ `mostrarDatos("Ana", 25);`

✗ `mostrarDatos("Ana");`

2 · Mismo orden

El primer argumento va al primer parámetro, el segundo al segundo…

✓ `mostrarDatos("Ana", 25);`

✗ `mostrarDatos(25, "Ana");`

3 · Tipo compatible

Si el parámetro es int, el argumento debe ser un número entero.

✓ `mostrarDatos("Luis", 30);`

✗ `mostrarDatos("Luis", "30");`

03

void y return

Métodos que solo hacen una tarea frente a métodos que, además, devuelven un resultado.

¿Qué significa void?

1. **void significa «vacío»:** el método NO devuelve ningún resultado al código que lo llamó.
2. **Aun así, hace cosas útiles:** mostrar mensajes, pintar un menú, pedir datos al usuario, recorrer una lista…
3. **Cuando necesitamos el resultado para seguir trabajando con él** (guardarlo, compararlo, sumarlo…), el método debe devolverlo con return.



Encargo tipo void

«Apaga la luz, por favor.»
La persona hace la tarea y no te trae nada de vuelta. No esperas ninguna respuesta.

```
apagarLuz();
```



Encargo con return

«¿Me dices qué hora es?»
La persona hace algo (mira el reloj) y te DEVUELVE una respuesta que tú usas después.

```
String hora = consultarHora();
```

Método que devuelve un valor

```java
public static int sumar(int a, int b) {
    int resultado = a + b;
    return resultado;
}

public static void main(String[] args) {
    int total = sumar(4, 6);
    System.out.println(total);
}
```

▶ SALIDA POR CONSOLA

```
10
```

1. **int antes del nombre** indica el TIPO de dato que devolverá el método (ya no es void).
2. **return resultado;** entrega el valor a quien llamó y termina el método en ese momento.
3. **La llamada se «sustituye» por el valor devuelto:** int total = sumar(4, 6); se convierte en int total = 10;
4. **Ahora main tiene el dato** guardado en total y puede usarlo como quiera.

sumar(4, 6) → a = 4, b = 6 → **return 10** → **total = 10**

¿Qué podemos hacer con un valor devuelto?

1. **Guardarlo en una variable**

```
int total = sumar(4, 6);
```

2. **Mostrarlo directamente**

```
System.out.println(sumar(4, 6));
```

3. **Usarlo en una operación**

```
int doble = sumar(2, 3) * 2;   // 10
```

4. **Usarlo en una condición**

```
if (sumar(a, b) > 100) { ... }
```

5. **Pasarlo a otro método**

```
mostrarResultado(sumar(4, 6));
```



Una llamada a un método con return se comporta como un valor: puedes ponerla en cualquier sitio donde pondrías un número o un texto.

Comparación: void frente a return

	Método void	Método con return
¿Qué pone antes del nombre?	`void`	`int, double, String, boolean…`
¿Devuelve un valor?	No. Solo realiza una tarea.	Sí, del tipo indicado.
¿Lleva return?	No es necesario.	Obligatorio, en todos los caminos posibles.
¿Cómo se suele llamar?	`saludar();`	`int t = sumar(4, 6);`
Uso típico	Mostrar mensajes, menús, pedir datos.	Cálculos, comprobaciones, conversiones.
¿Qué hace return?	—	Entrega el valor y termina el método inmediatamente.

Cuidado: mostrar NO es lo mismo que devolver

MUESTRA EL RESULTADO (void)

```
public static void mostrarDoble(int n) {
    System.out.println(n * 2);
}
```

El número aparece en la pantalla para que lo vea la persona usuaria, **pero el programa no se lo queda: se pierde.**

```
mostrarDoble(5);          // muestra 10
int x = mostrarDoble(5);  // ✗ ERROR
```

void no devuelve nada que se pueda guardar.

DEVUELVE EL RESULTADO (return)

```
public static int calcularDoble(int n) {
    return n * 2;
}
```

El número se entrega al programa, **que puede guardarlo, mostrarlo o usarlo en otros cálculos.**

```
int x = calcularDoble(5); // x vale 10
int y = x + 1;            // ✓ y vale 11
```

El valor se puede reutilizar.



println muestra el resultado a la persona; return se lo entrega al programa.

04

Combinar lo aprendido

Métodos junto con fórmulas, condicionales, bucles y Scanner.

Ejemplo: calcular el área de un triángulo

```java
public static double areaTriangulo(double base,
                                   double altura) {
    return (base * altura) / 2;
}

public static void main(String[] args) {
    double area = areaTriangulo(8, 5);
    System.out.println("Área: " + area);
}
```

▶ SALIDA POR CONSOLA

```
Área: 20.0
```

1. **double** porque el área puede tener decimales (por ejemplo, 4.5).
2. **Dos parámetros:** base y altura. Cada uno con su tipo.
3. **return directo:** podemos devolver una expresión sin guardarla antes en una variable.
4. **Se lee casi como una frase:** «el área es el área del triángulo de base 8 y altura 5».

Un ejercicio que ya conoces, ahora reutilizable.

Métodos y condicionales

```java
public static boolean esMayorDeEdad(int edad) {
    if (edad >= 18) {
        return true;
    } else {
        return false;
    }
}

System.out.println(esMayorDeEdad(20));  // true
System.out.println(esMayorDeEdad(15));  // false
```

▶ SALIDA POR CONSOLA

```
true
false
```

1. **boolean:** el método responde a una pregunta de sí o no (true / false).
2. **Hay dos caminos,** y cada uno tiene su propio return. Java exige que TODOS los caminos devuelvan algo.
3. **Muy útil dentro de un if:**

```java
if (esMayorDeEdad(edad)) {
    System.out.println("Puede votar");
}
```

Versión abreviada

La comparación ya es true o false:

```
return edad >= 18;
```

Métodos y bucles

```java
public static void mostrarNumeros(int limite) {
    for (int i = 1; i <= limite; i++) {
        System.out.println(i);
    }
}

// Desde main:
mostrarNumeros(5);
mostrarNumeros(3);
```

▶ SALIDA POR CONSOLA

```
1  2  3  4  5      ← mostrarNumeros(5)
1  2  3            ← mostrarNumeros(3)
```

(En consola, cada número aparece en su propia línea.)

1. **El parámetro limite** decide cuántas veces se repite el bucle.
2. **El bucle for** va desde 1 hasta el valor de limite, de uno en uno.
3. **La variable i** solo existe dentro del bucle: es local.
4. **Un mismo método, distintos resultados:** con 5 muestra cinco números; con 3, tres.

Ejemplo con Scanner

Doble.java

```java
import java.util.Scanner;

public class Doble {
  public static void main(String[] args) {
    Scanner teclado = new Scanner(System.in);
    System.out.print("Introduce un número: ");
    int numero = Integer.parseInt(teclado.nextLine());
    System.out.println(doble(numero));
    teclado.close();
  }

  public static int doble(int n) {
    return n * 2;
  }
}
```

▶ SALIDA POR CONSOLA

```
Introduce un número: 7
14
```

RECORRIDO DEL DATO

TECLADO el usuario escribe 7

↓

MAIN numero = 7

↓

LLAMADA doble(7) → n = 7

↓

DEVUELVE return 14

↓

MAIN println(14)

main pide y muestra; doble solo calcula.

Programa completo: main como director de orquesta

Notas.java · main

```
public static void main(String[] args) {
    Scanner teclado = new Scanner(System.in);

    System.out.print("Primera nota: ");
    double n1 = Double.parseDouble(teclado.nextLine());
    System.out.print("Segunda nota: ");
    double n2 = Double.parseDouble(teclado.nextLine());

    double media = calcularMedia(n1, n2);
    String nota = obtenerCalificacion(media);
    mostrarResultado(media, nota);

    teclado.close();
}
```

▶ SALIDA POR CONSOLA

```
Primera nota: 6.5    Segunda nota: 8
Media: 7.25 → Notable
```

CADA MÉTODO, UNA TAREA

`calcularMedia(n1, n2)`

double · return

Calcula y DEVUELVE la media.

`obtenerCalificacion(media)`

String · return + if

Decide y DEVUELVE el texto: Suspenso, Aprobado…

`mostrarResultado(media, nota)`

void

Solo MUESTRA el resultado final.

main no calcula: coordina a los demás métodos.

Programa completo: los métodos

Notas.java · métodos

```java
public static double calcularMedia(double a, double b) {
    return (a + b) / 2;
}

public static String obtenerCalificacion(double media) {
    if (media < 5) {
        return "Suspenso";
    }
    if (media < 7) {
        return "Aprobado";
    }
    return "Notable";
}

public static void mostrarResultado(double media, String nota) {
    System.out.println("Media: " + media + " → " + nota);
}
```

1. **calcularMedia** recibe dos notas y devuelve su media (double).

2. **obtenerCalificacion** recibe la media y devuelve un texto (String). Cuando una condición se cumple, return devuelve el texto y el método termina: las líneas siguientes ya no se ejecutan.

3. **mostrarResultado** recibe los dos datos y los muestra. No devuelve nada: es void.

Reto: añade «Sobresaliente» para medias de 9 o más.

05

Errores y práctica

Los fallos más habituales, cómo corregirlos y tres prácticas guiadas.

Errores habituales y cómo corregirlos

Error	✗ Así falla	✓ Así funciona
Olvidar los paréntesis al llamar	`saludar;`	`saludar();`
Argumento de tipo incorrecto	`sumar("4", 6);`	`sumar(4, 6);`
Declarar int y no devolver nada	`int sumar(...) { println(a+b); }`	`int sumar(...) { return a + b; }`
Guardar el resultado de un void	`int x = saludar();`	`saludar();`
Confundir println con return	Mostrar un valor no lo devuelve.	Usa return si necesitas el valor.
Declarar un método dentro de main	`main { … void saludar() {…} }`	Fuera de main, dentro de la clase.

¿Dónde se escribe un método?

✓ CORRECTO

```
public class MiPrograma {

    main(...) {

    saludar();

    }

    saludar() {

    println("Hola");

    }
}
```

✗ INCORRECTO

```
public class MiPrograma {

    main(...) {

        saludar() {
          println("Hola");
        }

    }
}
```

Los métodos van uno al lado del otro, nunca uno dentro de otro: dentro de la clase, pero fuera de main. Revisa bien que cada { tenga su }.

¿Detectas el error?

```
public static int multiplicar(int a, int b) {
    int resultado = a * b;
    System.out.println(resultado);
}
```

¿Por qué este código no compila?

¿Cómo podrías corregirlo? (hay dos formas)

Pistas

1. Fíjate en la palabra que hay justo antes del nombre del método.
2. ¿Qué «promete» ese int?
3. ¿Mostrar un número por pantalla es lo mismo que devolverlo?

Solución: el método promete un int… y no lo devuelve

Error del compilador: «missing return statement». Al escribir int, el método se compromete a devolver un número entero, pero solo lo muestra con println y no hay ningún return.

OPCIÓN A · CUMPLIR LA PROMESA: AÑADIR return

```
public static int multiplicar(int a, int b) {
    int resultado = a * b;
    return resultado;
}

// En main:
int r = multiplicar(3, 4);
System.out.println(r);   // 12
```

Elige A si vas a necesitar el resultado para otra cosa.

OPCIÓN B · NO PROMETER NADA: CAMBIAR A void

```
public static void multiplicar(int a, int b) {
    int resultado = a * b;
    System.out.println(resultado);
}

// En main:
multiplicar(3, 4);       // muestra 12
```

Elige B si solo quieres mostrarlo por pantalla.

Práctica 1 · Primeros métodos

Objetivo: separar la definición de un método de su llamada.

1. **Crea** un método mostrarBienvenida() que imprima un mensaje de bienvenida.
2. **Llámalo tres veces** desde main y comprueba que aparece tres veces.
3. **Crea otro método** mostrarDespedida() que imprima un mensaje de despedida.
4. **Cambia el orden** de las llamadas en main y observa cómo cambia la salida.
5. **Experimenta:** ¿qué ocurre si declaras un método pero nunca lo llamas?

PUNTO DE PARTIDA

Practica1.java

```
public class Practica1 {
    public static void main(String[] args) {
        // Escribe aquí las llamadas
    }

    // Declara aquí tus métodos
}
```

▶ SALIDA POR CONSOLA

```
¡Bienvenido/a al curso!
...
¡Hasta la próxima!
```

Práctica 2 · Parámetros

Objetivo: reutilizar un mismo método con distintos argumentos.

1. **Crea** el método mostrarTabla(int numero).
2. **Usa un bucle for** para imprimir su tabla de multiplicar del 1 al 10, con el formato «3 x 1 = 3».
3. **Llama al método** con los números 3, 5 y 7.
4. **Amplíalo:** añade un segundo parámetro, mostrarTabla(int numero, int hasta), para decidir hasta qué número llega la tabla.

PUNTO DE PARTIDA

Practica2.java

```
public static void mostrarTabla(int numero) {
    for (int i = 1; i <= 10; i++) {
        // Muestra: numero x i = resultado
    }
}
```

SALIDA POR CONSOLA

```
3 x 1 = 3
3 x 2 = 6
...   3 x 10 = 30
```

Práctica 3 · return

Objetivo: distinguir entre calcular, devolver y mostrar.

1. **Crea** calcularDescuento(double precio, double porcentaje).
2. **El método debe DEVOLVER** el precio final, sin mostrar nada. Fórmula: precio − precio × porcentaje / 100.
3. **Desde main,** pide el precio y el porcentaje con Scanner.
4. **Guarda el resultado** en una variable y muéstralo desde main.

PUNTO DE PARTIDA

Practica3.java

```
public static double calcularDescuento(
        double precio, double porcentaje) {
    // Calcula y devuelve el precio final
}

// Ejemplo: 80 € con un 25 % de descuento → 60.0
```

▶ SALIDA POR CONSOLA

```
Precio: 80
Descuento (%): 25
Precio final: 60.0
```

Repaso: responde sin ejecutar

1 Definir y llamar

¿Qué diferencia hay entre definir un método y llamarlo?

2 void

¿Qué significa que un método sea void?

3 Parámetro y argumento

¿Qué diferencia hay entre un parámetro y un argumento? Pon un ejemplo.

4 return

¿Cuándo conviene que un método devuelva un valor con return?

5 Varias llamadas

¿Se puede llamar varias veces al mismo método? ¿Qué ocurre cada vez?

6 ¿Qué muestra?

Si sumar devuelve a + b, ¿qué muestra…

```
println(sumar(3, 5));
```

RESUMEN

Lo esencial que debes recordar

Una tarea

Cada método realiza una tarea concreta y tiene un nombre que la describe.

main coordina

main dirige el programa llamando a los demás métodos.

Parámetros

Permiten que un método reciba datos y funcione con valores distintos.

return

Entrega un resultado al programa. Mostrar no es devolver.

Reutilizar

Menos repeticiones, código más claro y más fácil de mantener.

Siguiente paso: arrays y métodos que procesan muchos datos a la vez.